

TaskFlow 기능명세서

김상현 · 김정환

June 12, 2026

CONTENTS

TaskFlow 기능명세서	
1. 문서 개요	
1.1 공통 동작 원리	
2. 인증 (FR-26)	
3. 워크스페이스·보드·컬럼 (FR-01, FR-02)	
3.1 워크스페이스 생성	
3.2 보드 생성·이름변경·삭제 (FR-01)	
3.3 컬럼 추가·이름변경·삭제·순서변경 (FR-02)	
4. 카드 (FR-03, FR-04, FR-05)	
4.1 카드 생성 (FR-03)	
4.2 카드 수정·삭제 (FR-04)	
4.3 드래그 앤 드롭 이동 + 충돌 처리 (FR-05, FR-13)	
5. 필터·레이블·댓글·활동 (FR-06 ~ FR-09)	
5.1 필터·검색 (FR-06)	
5.2 레이블 (FR-07)·댓글 (FR-08)·활동 이력 (FR-09)	
6. 실시간 협업 (FR-10 ~ FR-13)	
7. GitHub 연동 (FR-14 ~ FR-18)	
7.1 저장소 연결 (FR-14)	
7.2 Webhook 처리 (FR-15 ~ FR-18)	

8. AI 자동 분류·추천 (FR-19 ~ FR-22)	
9. AI 회의록 (FR-23 ~ FR-25)	
9.1 회의 입력·받아쓰기 (FR-23)	
9.2 요약·할 일 추출 (FR-24) 및 카드 변환 (FR-25)	
10. 멤버·권한 관리 (FR-27)	
11. 데이터 모델 요약	
12. 테스트로 검증한 핵심 로직	

TaskFlow 기능명세서

1. 문서 개요

각 기능의 입력 → 처리 → 출력 과 화면/동작 흐름 을 기술한다. 모든 처리 단계는 실제 코드의 파일 경로·함수명을 근거로 적었으며, 명세는 현재 구현과 일치한다.

1.1 공통 동작 원리

- 읽기: React Server Component / Server Action 으로 Supabase에서 조회
- 쓰기(앱 내부): "use server" Server Action — Zod 검증 후 DB 반영, 경로 재검증(`revalidate-Path`)
- 외부 연동: Route Handler(`src/app/api/**`) — 웹훅·AI·STT
- 실시간 반영: Supabase Postgres CDC 구독(`use-board-realtime.ts`) → 디바운스 후 새로고침
- 낙관적 UI: Zustand 스토어(`entities/board/model/store.ts`)에 먼저 반영, 실패 시 스냅샷 복원
- 정렬: LexoRank 문자열(`shared/lib/lexorank.ts`)로 인접 두 항목만 계산해 위치 부여

2. 인증 (FR-26)

기능: 이메일/OAuth 로그인·회원가입.

구분	내용
진입 화면	<code>app/(auth)/login/page.tsx</code> , <code>app/(auth)/signup/page.tsx</code> → <code>features/auth-sign-in/ui/auth-form.tsx</code>
입력	이메일, 비밀번호, (가입 시) 표시명 / Google·GitHub 버튼
처리	Zod 검증(<code>model/schema.ts</code>) → Supabase Auth <code>signInWithPassword()</code> / <code>signUp()</code> / <code>signInWithOAuth()</code> . 보호 경로 접근 제어는 <code>src/middleware.ts</code>
출력	성공 시 <code>/workspaces</code> 이동, 비밀번호는 bcrypt 해시로 저장(Supabase), JWT 세션 발급. 실패 시 화면에 사유 표시
흐름	미로그인 사용자가 보호 경로 접근 → 미들웨어가 <code>/login</code> 으로 리다이렉트 → 로그인 성공 → 워크스페이스 목록
예외	OAuth 는 Redirect URL 미등록 시 비활성(데모는 이메일 로그인 사용)

3. 워크스페이스·보드·컬럼 (FR-01, FR-02)

3.1 워크스페이스 생성

구분	내용
진입	<code>features/workspace-create/ui/create-workspace-dialog.tsx</code>
입력	워크스페이스 이름(1~80자)
처리	<code>entities/workspace/api/actions.ts:createWorkspace()</code> — 이름 검증 → <code>work-spaces</code> insert → 생성자를 <code>owner</code> 로 <code>workspace_members</code> 자동 등록
출력	새 워크스페이스 id 반환, <code>/workspaces</code> 재검증

3.2 보드 생성·이름변경·삭제 (FR-01)

구분	내용
진입	생성 <code>features/board-create/...</code> , 편집/삭제 <code>features/board-edit/ui/board-menu.tsx</code>
입력	보드 이름(1~80자)
처리	생성 <code>createBoard()</code> — 보드 insert + 기본 컬럼 3개(To Do/In Progress/Done) 자동 생성. 이름변경 <code>entities/board/api/actions.ts:renameBoard()</code> , 삭제 <code>deleteBoard()</code> (cascade 로 컬럼·카드·댓글·레이블·활동 함께 삭제)
출력	생성 시 <code>/board/{id}</code> 진입, 삭제 시 워크스페이스로 복귀
권한	owner/admin 만 편집·삭제(RLS + UI 조건부)

3.3 컬럼 추가·이름변경·삭제·순서변경 (FR-02)

구분	내용
진입	<code>features/column-edit/ui/{add-column,column-header}.tsx</code>
입력	컬럼 이름(1~80자), (추가 시) 기준 위치
처리	<code>entities/column/api/actions.ts</code> — <code>createColumn()</code> (<code>position = between()</code>), <code>renameColumn()</code> (낙관적), <code>deleteColumn()</code> (카드 있으면 거부), <code>reorderColumn()</code>
출력	<code>columns</code> 추가/수정/삭제, 실시간 CDC 로 타 사용자 반영
미완	<code>reorderColumn</code> 액션·정렬 값은 구현, 컬럼 드래그 UI 연결만 미완 (카드 드래그와 충돌)

4. 카드 (FR-03, FR-04, FR-05)

4.1 카드 생성 (FR-03)

구분	내용
진입	<code>features/card-create/ui/add-card.tsx</code> (상세 옵션 토글, “시안 채우기” 데모 버튼)
입력	제목(1~200), 설명(선택), 우선순위(low/medium/high, 기본 medium), 마감일(선택), 대상 컬럼 id
처리	<code>entities/card/api/actions.ts:createCard()</code> — Zod 검증 → <code>shared/lib/append-position.ts:appendWithRetryReturning()</code> 로 컬럼 끝 위치 계산(동시 추가 충돌 시 재시도) → 카드 insert → <code>entities/board/api/activity.ts:logActivity()</code> 활동 기록
출력	카드 id, <code>board_activity</code> 기록, CDC INSERT → 타 사용자 화면 반영

4.2 카드 수정·삭제 (FR-04)

구분	내용
진입	<code>widgets/card-detail-panel</code> 카드 상세
입력	제목·설명·우선순위·마감일·담당자·레이블·댓글
처리	<code>updateCard()</code> , <code>deleteCard()</code> , <code>assignCard()/unassignCard()</code> — 변경 시 활동 메시지 (<code>entities/card/model/activity-message.ts</code>) 기록
출력	<code>cards</code> 갱신/삭제, 활동 피드 반영

4.3 드래그 앤 드롭 이동 + 충돌 처리 (FR-05, FR-13)

구분	내용
진입	<code>features/card-drag/model/use-card-dnd.ts</code> (DnD Kit <code>onDragEnd</code>)
입력	드래그한 카드, 드롭 위치(앞/뒤 카드·대상 컬럼)
처리	① 낙관적 이동(<code>store.moveCardLocal()</code>) → ② 이웃 위치로 새 <code>position</code> 계산 (<code>entities/card/model/move.ts:nextPosition()</code>) → ③ <code>moveCard()</code> 호출 시 <code>expected_updated_at</code> 과 현재 <code>updated_at</code> 비교(<code>hasConflict()</code>) → ④ 충돌 시 낙관적 UI 롤백 + 안내 후 새로고침
출력	<code>cards.position</code> · <code>column_id</code> 갱신, 컬럼 변경 시 활동 기록, CDC 로 타 사용자 반영
인수기준	먼저 화면에 반영하고, 다른 사용자가 먼저 바뀌 어긋나면 되돌려 일관성 유지

5. 필터·레이블·댓글·활동 (FR-06 ~ FR-09)

5.1 필터·검색 (FR-06)

구분	내용
진입	<code>features/board-filter/ui/board-filter-bar.tsx</code> (상단 고정)
입력	검색어, 우선순위, 담당자, 마감일(전체/지남/오늘/이번주/없음)
처리	<code>features/board-filter/model/filter.ts:filterCards()</code> — 조건 AND 결합, 마감일은 로컬 자정 기준 일 단위 비교(클라이언트 필터링)
출력	조건에 맞는 카드만 렌더, 활성 필터 배지 표시

5.2 레이블 (FR-07)·댓글 (FR-08)·활동 이력 (FR-09)

기능	처리	출력
레이블	<code>entities/label/api</code> — <code>createLabel/deleteLabel/assignLabel/unassignLabel</code> (<code>labels</code> , <code>card_labels</code>)	카드에 색상 레이블 표시
댓글	<code>entities/card/api/actions.ts</code> — <code>addComment/updateComment/deleteComment</code> . RLS(<code>0005</code>)로 본인 댓글만 수정·삭제	카드 상세에 댓글 스레드
활동 이력	<code>entities/board/api/activity.ts:logActivity()</code> , INSERT 정책 (<code>0007</code>) — owner/admin/member 만 기록	카드 생성·이동 등이 활동 피드에 자동 누적

6. 실시간 협업 (FR-10 ~ FR-13)

구분	내용
진입	<code>processes/realtime-collaboration</code> , <code>entities/board/model/use-board-real-time.ts</code> , <code>widgets/presence-cursors</code>
입력	(자동) 보드 입장, 마우스 이동, DB 변경 이벤트
처리	Supabase Realtime — Postgres CDC (cards/columns 변경 구독, 디바운스 resync 250ms), Presence (접속자), Broadcast (커서 좌표). 동시 수정 충돌은 4.3 참조
출력	한 사용자의 보드 변경이 타 사용자 화면에 즉시 반영, 접속자·커서 표시
디자인 패턴	Observer(변경 구독) + Memento(드래그 스냅샷/롤백)

7. GitHub 연동 (FR-14 ~ FR-18)

7.1 저장소 연결 (FR-14)

구분	내용
진입	<code>features/github-link-card/ui/{repo-form,copy-webhook-url}.tsx</code>
입력	<code>owner/repo</code> , (복사) 보드별 Webhook URL
처리	<code>entities/board/api/actions.ts:setBoardRepo()</code> — 형식 검증 후 <code>boards.github_repo</code> 저장. Webhook URL = <code>{SITE_URL}/api/webhooks/github?boardId={id}</code>
출력	보드-저장소 연결, GitHub Settings 에 등록할 URL/Secret 안내

7.2 WEBHOOK 처리 (FR-15 ~ FR-18)

구분	내용
진입 (엔드포인트)	<code>src/app/api/webhooks/github/route.ts</code> (POST)
입력	GitHub 페이로드 — <code>push</code> / <code>pull_request</code> (opened·closed·merged) / <code>issues</code> (opened)
처리	① 서명 검증 <code>shared/lib/github-signature.ts:verifySignature()</code> (HMAC-SHA256, timing-safe) → ② 역등 처리 <code>X-GitHub-Delivery</code> 중복 시 <code>skip(github_events)</code> → ③ 본문에서 카드 참조 추출(<code>TF-123</code> 등 정규식) → ④ PR 상태 전이 <code>features/pr-automation/model/transition.ts:resolvePrTransition()</code> → 대상 컬럼 매핑(<code>pick-TargetColumn()</code>) → ⑤ 카드 이동·배지·활동을 RPC 로 일괄 반영(동시 추가 충돌 시 재시도) → ⑥ 이슈 → 카드 자동 생성(FR-17)
출력	<code>github_events</code> 적재, <code>cards.github_url</code> · <code>position</code> · <code>github_pr_state</code> 갱신, “PR #N 머지됨.” 🚫 이슈 #N → 카드 생성” 활동 기록, CDC 로 실시간 반영
인수기준	같은 이벤트가 두 번 와도 한 번만 처리(역등), PR 머지 시 카드가 지정 컬럼으로 이동

8. AI 자동 분류·추천 (FR-19 ~ FR-22)

구분	내용
진입(엔드포인트)	<code>src/app/api/ai/classify/route.ts</code> (POST), UI <code>features/ai-classify-card/ui/classify-panel.tsx</code>
입력	카드 id (→ 제목·설명·마감일 자동 수집)
처리	① 카드·워크스페이스 조회 → ② 병렬로 멤버 목록(<code>fetchMembers</code>)·과거 확정 분류 예시(<code>fetchExamples</code> , <code>few-shot</code>) 수집 → ③ <code>shared/api/llm/index.ts:classifyTask()</code> 호출 → ④ 마감 임박 우선순위 가산(<code>shared/lib/ai-classify-logic.ts:bumpPriorityForDueDate()</code>) → ⑤ 추천 담당자 이름→id 매핑(<code>matchAssignee()</code> , 존재 멤버만)
출력 (JSON)	<code>{ category, priority, confidence, suggestedAssignee }</code> — 카테고리는 Bug/Feature/Refactor/Docs/Chore. 화면에 추천 표시, “적용” 시 카드에 반영, “자동 적용” 토글은 결과 즉시 반영
디자인 패턴	Strategy — LLM 공급자(<code>LLM_PROVIDER</code>) 교체 가능
흐름	카드 상세 → “AI 추천 받기” → 추천 카드(예: Bug · high · 신뢰도 100% · 담당자) → 사용자가 적용/거절(다음 추천에 반영)

9. AI 회의록 (FR-23 ~ FR-25)

9.1 회의 입력·받아쓰기 (FR-23)

구분	내용
진입	<code>features/meeting-record/ui/upload-form.tsx</code> , 페이지 <code>app/(main)/meetings/[meetingId]/page.tsx</code>
입력	워크스페이스, 제목, 참가자, 그리고 ① 실시간 회의(브라우저 Web Speech API, 화자 태그) ② 음성 파일 업로드 ③ 텍스트 직접 입력
처리	실시간은 <code>SpeechRecognition</code> (ko-KR) 중간 결과 누적. 저장 <code>entities/meeting/api/actions.ts:createMeeting()</code> — 오디오는 Supabase Storage 업로드, <code>meetings</code> <code>insert(status='pending')</code> . 음성 파일 STT 는 <code>src/app/api/meetings/transcribe/route.ts</code> → <code>shared/api/stt:transcribeAudio()</code> → <code>meetings.transcript</code>
출력	<code>meetings</code> 레코드, <code>/meetings/{id}</code> 진입

9.2 요약·할 일 추출 (FR-24) 및 카드 변환 (FR-25)

구분	내용
진입 (엔드포인트)	<code>src/app/api/meetings/extract-tasks/route.ts</code> (POST), UI <code>features/meeting-to-cards/ui/action-items.tsx</code>
입력	회의 id (→ transcript)
처리	① <code>shared/api/llm/index.ts:summarizeMeeting()</code> — 요약·참석자·안건·논의·결정·할 일 추출 → ② <code>meetings.summary.structured (jsonb).status='done'</code> 저장, <code>meeting_action_items</code> insert → ③ 사용자가 항목 제목·담당자 편집 후 대상 컬럼 선택 → ④ <code>entities/meeting/api/actions.ts:createCardFromActionItem()</code> 로 카드 생성, <code>meeting_action_items.card_id</code> 연결
출력	구조화 회의록 + 카드 생성. 흐름: 회의 입력 → “작업 추출” → 할 일 N개 → “카드로 보내기”

10. 멤버·권한 관리 (FR-27)

구분	내용
진입	<code>features/member-manage/ui/invite-member-form.tsx</code> , 페이지 <code>app/(main)/work-spaces/[id]/page.tsx</code>
입력	초대 이메일, 역할(member/admin — owner 배정 불가)
처리	초대 <code>entities/workspace/api/actions.ts:inviteMember()</code> → RPC <code>invite_member_by_email()</code> (<code>0008</code> , SECURITY DEFINER, 권한·중복 검사). 역할 변경 <code>changeMemberRole()</code> · 제거 <code>removeMember()</code> — 권한 검사 + 마지막 owner 강등/삭제 방지(<code>model/member-permissions.ts:wouldRemoveLastOwner()</code>)
출력	<code>workspace_members</code> 추가/변경/삭제. 검증 실패는 throw 가 아닌 <code>{ ok, message }</code> 결과로 반환해 화면에 안내
권한강제	모든 테이블 RLS (<code>0002</code>) 적용 — 화면을 우회해도 DB 단계에서 차단. 멤버십 헬퍼는 SECURITY DEFINER 로 정책 재귀 방지

11. 데이터 모델 요약

테이블	핵심 컬럼	관련 FR
<code>user_profiles</code>	id, email, display_name (auth.users 미리, 가입 트리거)	FR-26
<code>workspaces</code> / <code>work-space_members</code>	name / user_id, role(owner·admin·member)	FR-27
<code>boards</code>	name, github_repo, pr_automation_enabled, pr_open/merged_column_id	FR-01·14·16
<code>columns</code>	board_id, name, position(LexoRank)	FR-02
<code>cards</code>	column_id, title, description, priority, due_date, ai_category, github_url, github_pr_state, updated_at	FR-03·05·19
<code>labels</code> / <code>card_labels</code>	name, color / 매핑	FR-07
<code>comments</code>	card_id, author_id, body	FR-08
<code>board_activity</code>	board_id, card_id, message	FR-09
<code>meetings</code> / <code>meeting_acion_items</code>	transcript, summary, structured / title, suggested_assignee, card_id	FR-23~25
<code>github_events</code>	delivery_id, event_type, payload	FR-15~18

12. 테스트로 검증한 핵심 로직

`tests/` 의 단위 테스트 17개 파일이 전부 통과(`npm test`)하며, 다음을 검증한다: LexoRank 정렬, 카드 이동 충돌 감지, AI 분류 로직(마감일 가산·담당자 매칭), PR 상태 전이, GitHub 서명 검증, 회의 요약 파싱, 카드 필터, 멤버 권한(마지막 owner 보호).